Interfícies

Índex

Interfícies....1
1. Sintaxis....2
2. Diagrama de classes....3
3. Implementant interfícies....4
4. Exemple Vehicle....8
5. Variables a les interfícies....10
6. Les interfícies es poden estendre....10
7. Implementació de mètodes a subclasses....11
8. Herència múltiple....11
9. Interfícies - Polimorfismes....12
10. Utilització d'interfícies....14

Com hem vist anteriorment les classes **abstractes** només defineixen el que fan, però no com ho fan, deixant a les classes concretes la seva implementació. Les classes abstractes contenen mètodes abstractes però poden tenir també mètodes no abstractes.

Les interfícies són classes (sense atributs ni constructor) amb constants i mètodes abstractes*. I les classes concretes han d'implementar tots els mètodes.

Les classes que utilitzin o implementin les interfícies estan obligades (tenen un contracte) a implementar els seus mètodes.

*** A partir de la versió 8 les interfícies poden tenir també mètodes default i static i a partir de la versió 9 mètodes private.**

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

1. Sintaxis

- Utilitzem la paraula reservada **interface**
- Han de ser **publiques** o **visibilitat package** (default)
- No tenen atributs ni constructor
- No es poden instanciar
- Solen acabar en **-able**: *Comparable*, *Imprimable*, *Movable*, etc.
- Les constants, totes són declarades com **public static final** (podem ometre aquestes paraules reservades)
- Els mètodes **abstract** podem prescindir de la paraula **public abstract**
- Pot contenir mètodes **public static** (a partir de la versió 8), podem prescindir de la paraula **public**
- pot contenir mètodes **public default** (a partir de la versió 8), podem prescindir de la paraula **public**
- pot contenir mètodes **private** (a partir de la versió 9)

Això:	És el mateix que:

`public interface MyInterface {`
`  //Declare constants`
`  public static final int MAX = 10;`
 	
`  //declare abstract methods`
`  public abstract void method1();`
 	
`  //declare default methods  // from Java 8`
`  public default int method2() {...}`
 	
`  //declare static methods  // from Java 8`
`  public static int method3() {...}`
 	
`  //declare private methods // from Java 9`
`  private int method4() {...}`
`}`
	`public interface MyInterface {`
	`  //Declare constants`
	`  int MAX = 10;`
	
	`  //declare abstract methods`
	`  void method1();`
	
	`  //declare default methods  // from Java 8`
	`  default int method2() {...}`
	
	`  //declare static methods  // from Java 8`
	`  static int method3() {...}`
	
	`  //declare private methods  // from Java 9`
	`  private int method4() {...}`
	`}`

Els mètodes abstractes d'una interfície han de ser tots públics, en canvi, les interfícies poden ser públiques o amb visibilitat **package** (default, tan sols visibles al mateix package).

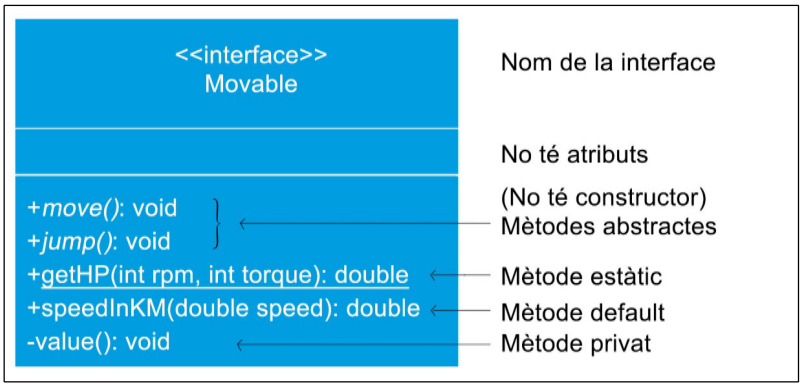
Interface Public:	Interface package:

`public interface MyInterface {`
 	
`}`
	`interface MyInterface {`
	
	`}`

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

2. Diagrama de classes

El diagrama de classes UML ens mostra la paraula **<<interface>>** a la part superior.
Els **mètodes abstractes** estan en cursiva.
Els **mètodes estàtics** ens mostren subratllats.
Els **mètodes default** es mostren sense cap modificació.



Exemple de representació d'una Interfície

Mètodes abstractes

Els **mètodes abstractes** només estan declarats a la interfície, conté la la signatura del mètode i un punt i coma al final, és a dir, descriu el què fa, però no com ho fa, les classes que implementin aquesta interfície hauran de sobreescriure aquests mètodes.

`void move();` `void jump();`	`public abstract void move();` `public abstract void jump();`

Mètodes default

Els mètodes **default** que incorpora la interfície, permet tenir un mètode que és comú a totes les classes que l'implementen i no haver de sobreescriure aquest mètode a cada una de les classes. Abans de la versió 8, si s'afegia un mètode a una interfície, aquest havia de ser **abstract**, i totes les classes que l'implementaven havien de sobreescriure aquest mètode, això podia implicar molts canvis, i per tant amb aquest nou mètode **default** evitem aquest problema.
Les subclasses poden sobreescriure els mètodes **default**, en aquest cas, la classe ha de canviar la signatura del mètode, no pot tenir mètodes **default**, ha de definir el mètode amb un modificador d'accés sempre superior o igual al definit a la interfície.

`default double speedInKMH(double speed) {..}`	`public default double velocity(double speed) {..}`

Mètodes estàtics

Els mètodes **estàtics** d'una interfície, s'utilitzen igual que els mètodes estàtics de les classes, són independents de l'objecte, i es criden amb el nom de la interfície al davant. Al ser estàtics, no té molt de sentit que les classes el sobreescriguin, però Java8 permet sobreescriure'ls.

`static void getHP(int rpm, int torque) {..}`	`public static void getHP(int rpm, int torque) {...}`

Mètodes privats

Els mètodes **privats** d'una interfície, a partir de la versió Java 9, s'utilitzen per ser cridats només pels mètodes de la interfície, i no fora d'ella.

```
private int value() {...}
```


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

3. Implementant interfícies

Tenim dues maneres d'implementar les interfícies.

Opció 1:

Utilitzant la paraula reservada **implements**. Un cop s'ha definit una interfície, podem fer que una o més classes **implementin** aquesta interfície. El fet d'implementar una interfície, igual que amb les classes abstractes, obliga a la classe a implementar els mètodes abstractes definits en ella.

Per exemple, la classe **Car** implementa la interfície **Movable**:

Interface **Movable**:

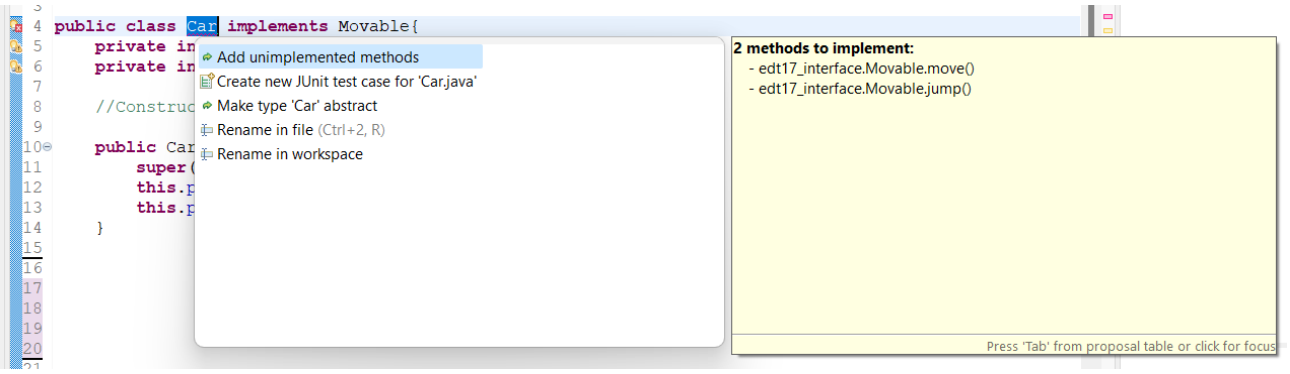
```java
public interface Movable {
    //Constants
    double HP = 5225;
    double FACTOR = 3.6;

    //abstract methods
    void move();

    void jump();

    //static method
    static double getHP(int rpm, int torque) {
        return (rpm * torque) / HP;
    }

    //default method
    default double speedInKMH(double speed) {
        return speed * FACTOR;
    }
    //private method
    private double value() {
        return HP * 1000;
    }
}
```

Classe **Car** implementa **Movable**:

```java
public class Car implements Movable {
    private int posX;
    private int posY;
    //Constructors
    ...

    @Override
    public void move() {
        posX++;
    }
    @Override
    public void jump() {
        posY++;
    }

    //optional
    static double getHP(int rpm, int torque) {
        return (rpm * torque) / 5.2;
    }

    //optional
    @Override
    public double speedInKMH(double speed) {
        return speed * FACTOR;
    }

}
```

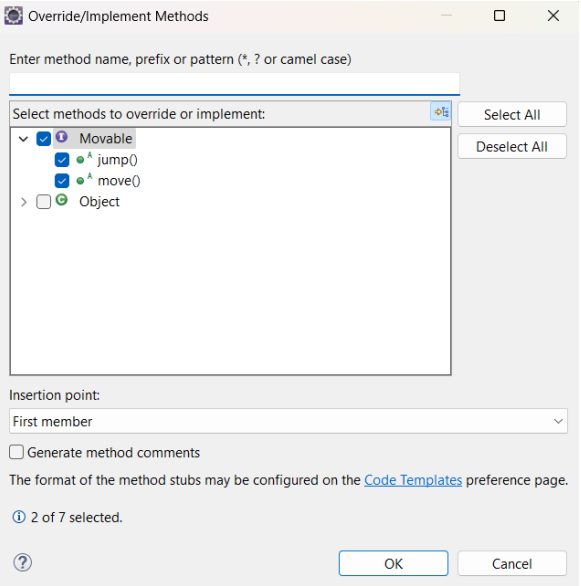
La classe també pot sobreescriure si vol els mètodes **estàtics** i **default** si és necessari, però no és obligatori. Notar que els mètodes a la classe **Car** s'han de definir amb la mateixa visibilitat o superior però mai inferior.

Amb l'ajuda d'Eclipse es poden implementar els mètodes abstractes de la interfície:
Fent clic a l'error de compilació, ens mostra el menú de les opcions per corregir l'error, en aquest cas, la primera opció "*Add unimplemented methods*" ens afegirà els dos mètodes abstractes de la interfície.



Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

També es pot realitzar el mateix a través del menú:
Source > Override / Implement methods ...



Seleccionar els dos mètodes abstractes i fer clic a "*OK*".

Exemple de com utilitzar els mètodes en el **main**:

```
Car a = new Car(0, 0);

//abstract method
a.move();
a.jump();

//using interface constants
double b = Movable.HP;

//static method
Movable.getHP(2000, 120);

//default method
a.speedInKMH(20);
```


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Opció 2:
Sense utilitzar la paraula **implements**, podem instanciar la interfície i al mateix temps sobreescriure els mètodes abstractes:

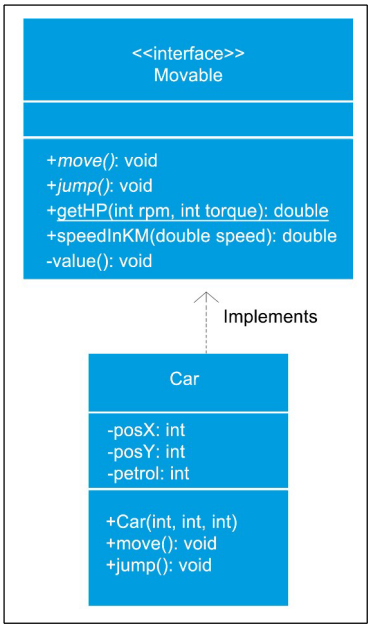
```
public interface Movable {
        void move();

        void jump();

}
```

```
public class Car{
    private int posX;
    private int posY;
    //Constructors
    ...

    Movable y = new Movable() {

        @Override
        public void move() {
                posX++;

        }

        @Override
        public void jump() {
                posY++;

        }
    };
}
```

Relació Interfície-Classe

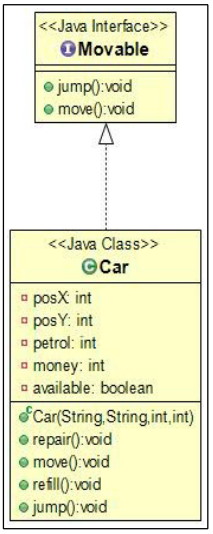
Les classes que implementen aquesta interfície es mostra amb una fletxa discontinua.

```
public class Car implements Movable { ...}
```

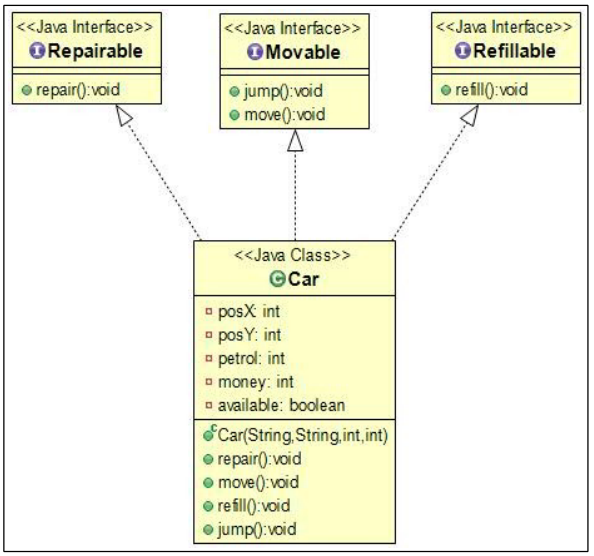


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Implementació simple i múltiple:



Simple



Múltiple

```
public class Car implements Movable { ...}
```

```
public class Car implements Movable, Repairable, Refillable { ... }
```

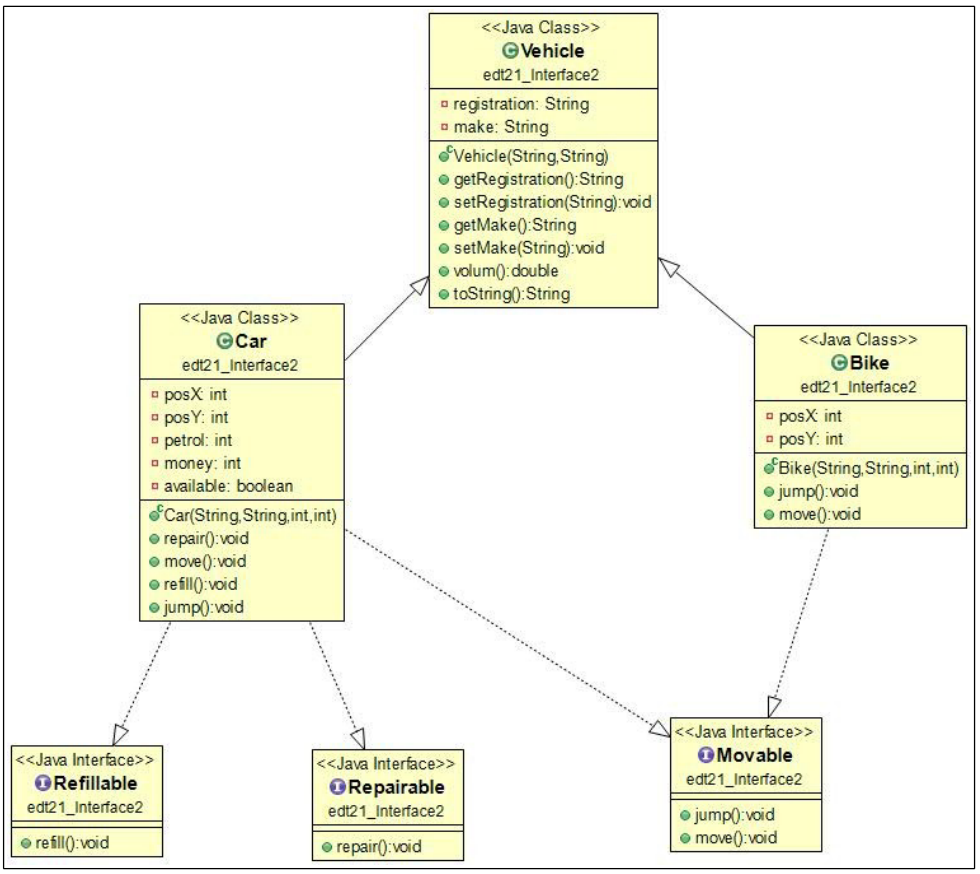

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

4. Exemple Vehicle

El principal avantatge de les interfícies és que una classe pot implementar més d'una interfície. A més, una classe pot estendre a una altre classe i implementar una, dos o les interfícies que sigui necessari. Les interfícies a implementar han d'anar al final de la declaració de classe i separades per comes.

La classe **Car** i **Bike** hereten de **Vehicle**, però cada classe té unes propietats diferents que venen donades per les interfícies que implementen, per exemple, la bicicleta i el cotxe es poden moure, i per tant implementen la interfície **Movable**, però de diferent manera, per això cada classe sobreescriu el mètode **jump()** i **move()**.

El cotxe a més pot reparar-se i pot reomplir el dipòsit de benzina i per això implementa les interfícies **Repairable** i **Refillable**.



Definim les interfícies amb els seus mètodes abstractes que les subclasses hauran d'implementar.

```
public interface Movable {
    void jump();
    void move();
}
```

```
public interface Repairable {
    void repair();

}
```

```
public interface Refillable {
        void refill();

}
```


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

La classe **Bike** hereta de **Vehicle** i implementa la interfície **Movable** i **Repairable**, per tant haurà de definir com es mou i salta, i com es repara la bicicleta:

```
public class Bike extends Vehicle implements Movable, Repairable {
        private int posX;
        private int posY;

        public Bike(String registration, String make, int posX, int posY) {
                super(registration, make);
                this.posX = posX;
                this.posY = posY;
        }
        @Override
        public void jump() {
                posY++;
        }
        @Override
        public void move() {
                posX++;
        }
        @Override
        public void repair() {
                //TODO
        }
}
```

La classe **Car** hereta de **Vehicle** i implementa la interfície **Movable**, **Repairable** i **Refilable**, per tant haurà de definir com es mou i salta, com es repara i com aconsegueix gasolina el cotxe:

```
public class Car extends Vehicle implements Movable, Reparable, Refillable {
        private int posX;
        private int posY;

        public Car(String registration, String make, int posX, int posY) {
                super(registration, make);
                this.posX = posX;
                this.posY = posY;
        }
        @Override
        public void move() {
                posX++;
        }
        @Override
        public void jump() {
                posY += 2;

        }
        @Override
        public void repair() {
                //TODO
        }
        @Override
        public void refill() {
                //TODO
        }
}
```


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

5. Variables a les interfícies

Les variables d'una interfície són necessàriament públiques, estàtiques i finals (podem ometre `public static final`). Això és una limitació, però en certes situacions pot ser útil. Per exemple, de vegades un programa necessita definir un conjunt de constats. Una manera de fer-ho és mitjançant una interfície:

Definició de constants:	Definició de constants sense ometre `public static final`:

`public interface Constants {`
`  int MIN = 0;`
`  int MAX = 100;`
`  String ERROR_MSG = "Error!";`
`}`
	`public interface Constants {`
	`  public static final int MIN = 0;`
	`  public static final int MAX = 100;`
	`  public static final String ERROR_MSG = "Error!";`
	`}`

Per a fer servir aquestes variables hi ha dues opcions. La primera és implementant aquesta interfície, i les constants es poden utilitzar directament (com si s'haguessin definit a la mateixa classe).

La segona opció, és **no implementant-la**, en aquest cas, les constants es fan servir de de la mateixa manera que les faríem servir si són definides a una classe normal:

Implementant la interfície	No implementant la interfície

`public class Program implements Constants {`
`  public static void main(String[] args) {`
`  int[] array = new int[MAX];`
`  //...`
`  }`
`}`
	`public class Program {`
	`  public static void main(String[] args) {`
	`  int[] array = new int[Constants.MAX];`
	`  //...`
	`  }`
	`}`

6. Les interfícies es poden estendre

Una interfície pot estendre a una altra interfície (o a més d'una):

La interfície **B** hereta de **A**:

`public interface A {` `}`	`public interface B extends A {` `}`

Si una classe **C** implementa una interfície **B** que a la vegada aquesta interfície hereta d'una altra **A**, la classe haurà d'implementar tots els mètodes, els de la interfície **A** i **B**.

`public interface A {`
`  public int method1();`
`}`
	`public interface B extends A {`
	`  public int method2();`
	`  public int method3();`
	`}`
		`public class C implements B {`
		`  public int metode1() {`
		`  return 0;`
		`  }`
		`  public int metode2() {`
		`  return 0;`
		`  }`
		`  public int metode3() {`
		`  return 0;`
		`  }`
		`}`

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

7. Implementació de mètodes a subclasses

Ja hem vist que una classe que implementa una interfície ha d'implementar tots els seus mètodes. Malgrat tot, pot ser que que no hagi d'implementar-los tots, com en el cas de que la classe sigui heretada per una altra classe, aleshores la subclasse hauria d'implementar els que no ha implementat la superclasse.

```
public interface Y {
        public void X1();
        public void X2();
        public void X3();
}
```

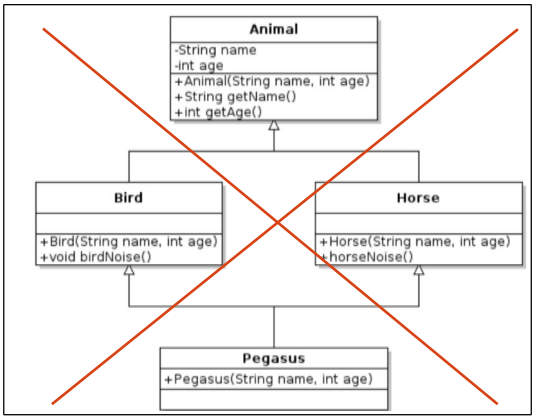
```
public abstract class A implements Y {
        @Override
        public void X1() {
            // TODO
        }
        @Override
        public void X2() {
            // TODO
        }
}
```

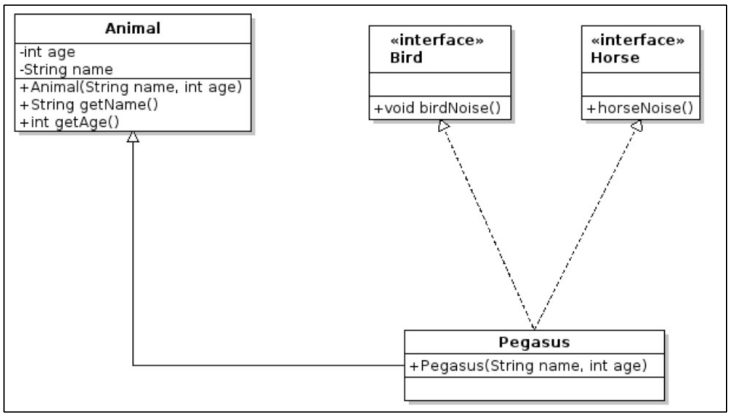
```
public class B extends A {
        @Override
        public void X3() {
            // TODO
        }
}
```

La classe A ha de ser abstracta perquè no implementa tots els mètodes, en aquest cas el mètode X3, i la subclasse B haurà d'implementar el mètode X3 que no implementa la classe A.

8. Herència múltiple

Com ja hem dit, Java no permet l'herència múltiple com si ho fan altres llenguatges com C++. Per exemple, la classe **Pegasus** hereta de **Bird** i **Horse**, per poder realitzar aquesta jerarquia, ho podem aconseguir mitjançant interfícies.





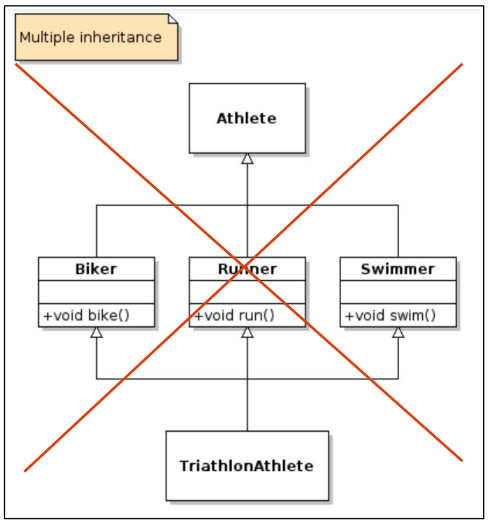
Herència múltiple no és possible a Java	Solució per l'herència múltiple

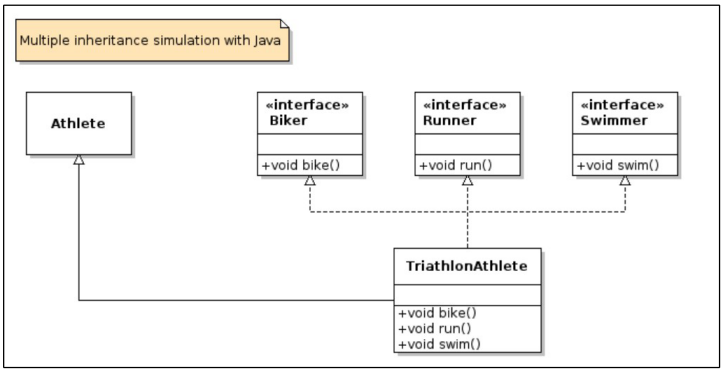
~~`public class Pegasus extends Bird, Horse {`~~
~~`...`~~
~~`}`~~
	`public class Pegasus extends Animal implements Bird, Horse{`
	`...`
	`}`

Quan implementem interfícies, estem afegint mètodes a les classes que l'implementen, però no afegim nous atributs, és per això que l'herència múltiple en Java no és totalment una herència múltiple com a altres llenguatge.

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Exemple: Atleta triatló
Un atleta que practica triatló, podria heretar de 3 classes (**Biker**, **Runner** i **Swimmer**), però en Java l'herència múltiple no es permet, la simulació per aquest cas podria ser la següent:





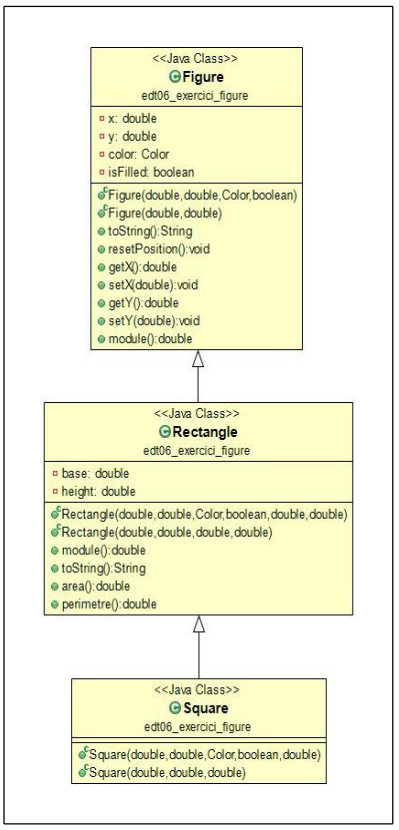
9. Interfícies - Polimorfismes

La interfície és un tipus de dades, per tant podem utilitzar-la per declara referències o en arguments de mètodes.
Igual com passava amb les classes, els polimorfisme entre classes permet utilitzar les superclasses per referenciar objectes de les subclasses (el que s'anomena conversió cap a munt).

Exemple de polimorfisme entre classes:

Aquí tenim un exemple de polimorfisme d'herència, els objectes **Square** es poden definir des de **Figure**, **Rectangle** o **Square**:

```
Figure f = new Square(...);
Rectangle f = new Square(...);
Square f = new Square(...);
```



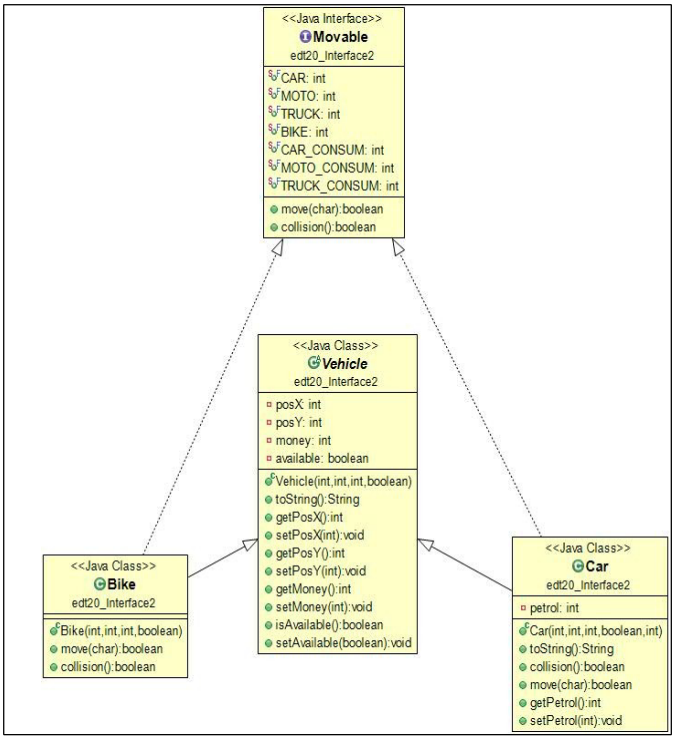
Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Exemple de polimorfisme en interfícies:

De manera similar, les interfícies es poden utilitzar per referenciar objectes que pertanyen a les classes que implementen aquestes interfícies. Això permet el polimorfisme en temps d'execució, tal i com ho fèiem amb les classes abstractes.

Imaginem que tenim definida aquesta interfície **Movable** amb un mètode abstracte **move()**:

```
public interface Movable {
    boolean move(char pos);
}
```



En el següent codi mostra com podem treballar amb la referència **vehicle**, sense la necessitat de saber quin tipus d'objecte és amb en aquesta referència.

```
Movable vehicle;

if (condition ...) {
        vehicle = new Car(0, 0, 1000, true, 100);
} else {
        vehicle = new Bike(0, 0, 1000, true);
}
vehicle.move('x');
```

I durant l'execució, quan es crida el mètode abstracte **move()** de la interfície, s'escollirà el mètode adequat depenent del tipus d'objecte (**Car** o **Bike**).

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

10. Utilització d'interfícies

Si volem ampliar un projecte sense tocar o canviar les classes i interfícies existents, tenim l'opció de crear noves classes que utilitzin les nostres interfícies com objectes.

Per exemple, tenim una interfície **Movable** que conté els dos mètode abstractes **move** i **jump**, per moure's en l'eix de les x i en l'eix de les y respectivament.

```
public interface Movable {
        void move();
        void jump();

}
```

La classe abstracta **Vehicle**, conté els atriubuts **posX** i **posY**, i la nostra classe **Car** hereta de **Vehicle** i implementa la interfície **Movable**:

```
public abstract class Vehicle {
    private int posX;
    private int posY;
    public Vehicle(int posX, int posY) {
        this.posX = posX;
        this.posY = posY;
    }
}
```

```
public class Car extends Vehicle implements Movable{
    private String make;
    private String model;
    public Car(int posX, int posY, String make, String model) {
        super(posX, posY);
        this.make = make;
        this.model = model;
    }
    @Override
    public void move() {
        setPosX(getPosX() + 1);

    }
    @Override
    public void jump() {
        setPosY(getPosY() + 1);

    }
}
```

Ens demanen que els nostres objectes puguin moure's també en diagonal, si afegim un nou mètode abstracte (**moveXY**) dins de la interfície **Movable**, haurem de sobreescriure aquest mètode a totes les classes que l'implementen, i això podria ser un problema, hauríem de provar totes les classes de nou, o fins i tot, potser no tenim accés per canviar aquestes classes. Una de les solucions, és crear una classe que utilitzi la interfície **Movable** com objectes.

Podem realitzar aquesta ampliació de dues maneres, utilitzant mètodes estàtics o utilitzant mètodes no estàtics.

La classe **Diagonal** utilitza interfícies com objectes i la classe **Test** mostra com podem utilitzar la nova classe **Diagonal**. Tenim dues maneres d'implementar aquesta solució, amb un mètode d'instància (no estàtic) o un mètide estàtic:

Utilitzant mètodes no estàtics

```
public class Diagonal {
        void moveXY(Movable m) {
                m.move();
                m.jump();
        }
}
```

```
public class Test {
        public static void main(String[] args) {
                Movable car = new Car(0, 0, "SEAT", "IBIZA");
                Diagonal diagonal = new Diagonal();
                diagonal.moveXY(car);
                System.out.println(car.toString());
        }

}
```


Copyright (C) Joan Sèculi (joan.seculi@gmail.com)

Utilitzant mètodes estàtics:

```java
public class Diagonal {
   static void moveXYStatic(Movable m) {
        m.move();
        m.jump();
   }
}
```

```java
public class Test {
   public static void main(String[] args) {
        Movable car2 = new Car(0, 0, "SEAT", "IBIZA");
        Diagonal.moveXYStatic(car2);
        System.out.println(car.toString());
   }
}
```

Llicència

Copyright (C) Joan Sèculi (joan.seculi@gmail.com)
Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is in http://www.gnu.org/licenses/fdl.txt